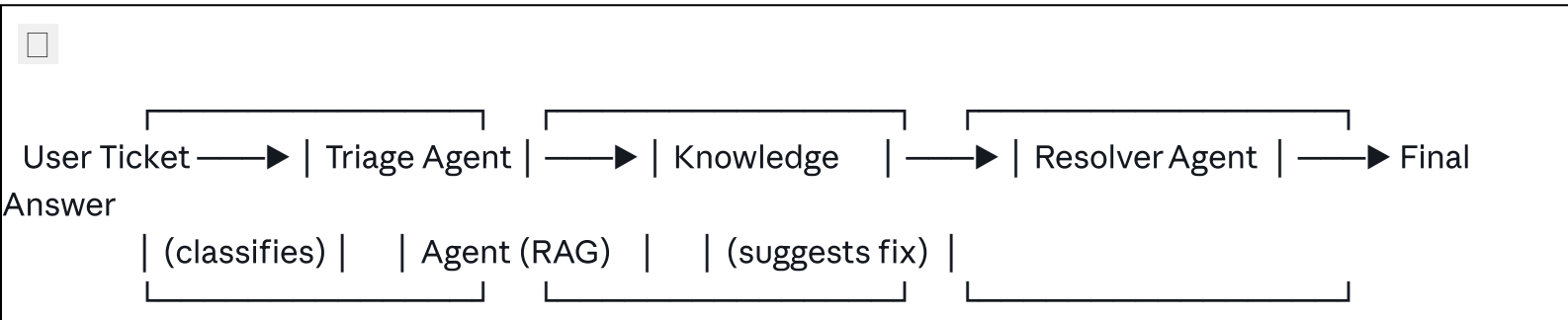


Enterprise IT Helpdesk Multi-Agent System

A multi-agent AI system that automates the first line of IT helpdesk triage — classifying incoming tickets, retrieving relevant knowledge base documentation, and suggesting a grounded first troubleshooting step. Built as a hands-on learning project to explore agentic AI architecture patterns used in real enterprise systems.

Architecture

The system uses a **3-agent pipeline** orchestrated with LangGraph, where each agent reads from and writes to a shared state object as the ticket flows through the system:



Triage Agent — reads the raw ticket description and classifies it into one of six categories (Password Reset, Hardware Issue, Software Bug, Network Issue, Access Request, Other).

Knowledge Agent — performs Retrieval Augmented Generation (RAG): converts the ticket into a vector embedding and searches a ChromaDB vector store for the most relevant internal knowledge base document, rather than relying on the LLM's raw training data.

Resolver Agent — combines the ticket, its category, and the retrieved knowledge to suggest one concrete, grounded first troubleshooting step.

The pipeline is exposed two ways:

Streamlit UI (`ui/app.py`) — a browser-based form for a human to submit a ticket and see results.

FastAPI backend (`api/main.py`) — a REST endpoint (`POST /submit-ticket`) so other systems (a ticketing tool, a chatbot, Copilot Studio, etc.) can call the pipeline programmatically.

Tech Stack

Component	Technology
LLM	Ollama (llama3.2), running locally
LLM orchestration	LangChain
Multi-agent workflow	LangGraph
Vector database / RAG	ChromaDB

Component	Technology
Web UI	Streamlit
REST API	FastAPI + Uvicorn
Language	Python 3.12+

Project Structure

```

├── enterprise-it-helpdesk-multiagent/
│   ├── agents/           # Individual agent logic
│   │   ├── triage_agent.py
│   │   ├── knowledge_agent.py
│   │   └── resolver_agent.py
│   ├── graphs/
│   │   └── helpdesk_graph.py # LangGraph pipeline wiring the 3 agents together
│   ├── data/             # Sample IT knowledge base documents (.txt)
│   ├── ui/
│   │   └── app.py         # Streamlit web interface
│   ├── api/
│   │   └── main.py        # FastAPI REST endpoint
│   ├── tests/            # (planned) automated tests
│   ├── requirements.txt
│   └── NOTES.md          # Personal learning notes / concept cheat-sheet
```

Setup & Installation

Prerequisites: Python 3.12+, Git, [Ollama](#) installed with the `llama3.2` model pulled (`ollama pull llama3.2`).

```

bash
```

```
# Clone the repo
git clone https://github.com/Gundam-Rajendar4/enterprise-it-helpdesk-multiagent.git
cd enterprise-it-helpdesk-multiagent
```

```
# Create and activate a virtual environment
```

```
python -m venv venv
venv\Scripts\activate    # Windows
# source venv/bin/activate # Mac/Linux
```

```
# Install dependencies
pip install -r requirements.txt
```

Usage

Run the Streamlit UI:

```

bash
streamlit run ui/app.py
```

Opens at — type a ticket description and click Submit.

Run the FastAPI backend:

```

bash
uvicorn api.main:app --reload
```

Interactive API docs available at .

Example request:

```

bash
curl -X POST http://127.0.0.1:8000/submit-ticket \
-H "Content-Type: application/json" \
-d '{"ticket": "I forgot my password and cannot log into my laptop."}'
```

Run the pipeline directly from the terminal:

```

bash
python -m graphs.helpdesk_graph
```

Known Limitations

This is a learning-stage project, not a production system. Current gaps:

No confidence threshold on RAG retrieval — the Knowledge Agent will always return its "closest" match even for unrelated queries, since ChromaDB has no built-in way to say "nothing relevant found."

No error handling for edge cases (empty input, Ollama not running, malformed requests).

No automated tests yet.

Knowledge base is a small set of 4 sample documents — not representative of a real enterprise KB.

No authentication/security on the API endpoint.

No integration with an actual ticketing system (e.g., ServiceNow) — the system suggests a resolution but doesn't create or update real tickets.

Runs locally only; not yet deployed anywhere publicly accessible.

Planned Improvements

Add confidence scoring to the RAG retrieval step

Add error handling and input validation

Write automated tests (`tests/` folder)

Expand knowledge base and add a document-ingestion pipeline

Add conditional routing in LangGraph (e.g., route "Access Request" tickets differently from "Hardware Issue")

Deploy publicly (Streamlit Community Cloud / cloud hosting)

Author

Built by Rajendar Gundam as a hands-on project to learn agentic AI system design — LangGraph orchestration, RAG, and multi-agent architecture patterns applicable to enterprise AI platforms.